

CD Lab Record 2

Title : Design and Implementation of a Lexical Analyzer for a Toy Language

Objective

The objective of this experiment is to design and implement a Lexical Analyzer for a given toy programming language. The lexical analyzer is the first phase of a compiler and is responsible for reading the source program character by character, grouping characters into meaningful units called lexemes, and converting them into tokens. Through this experiment, the concepts of tokenization, pattern matching using transition diagrams, and symbol table construction are clearly understood.

Theory

A Lexical Analyzer (Scanner) acts as an interface between the source program and the syntax analyzer. Its main functions include:

- Removing whitespace, tabs, and comments
- Recognizing valid tokens such as keywords, identifiers, constants, operators, and delimiters
- Reporting invalid or unrecognized tokens
- Maintaining a symbol table for identifiers

Each token is represented in the form: `<TOKEN_CLASS, LEXEME>`

The recognition of tokens is based on finite automata derived from regular expressions. Transition diagrams are used to visually represent the movement between states while reading characters from the input stream.

Language Specification

The toy language supports:

- Alphabets: A–Z, a–z
- Digits: 0–9
- Special Symbols: ; , () { } []

Keywords: `if, else, while, int, float, return, begin, end, print`

Operators: `+, -, *, /, =` Relational Operators: `<, >, ≤, ≥, =, ≠`

Identifiers: Defined by the rule: `letter (letter | digit)*` Constants:

- Integer constants (e.g., 10, 123)
- Floating-point constants (e.g., 12.34, 0.5)

Design of Transition Diagrams

Separate transition diagrams were designed for the following token categories:

1. Identifiers / Keywords- Start with a letter- Followed by letters or digits- After recognition, lexeme is checked against the keyword list
2. Integer Constants- One or more digits- No decimal point allowed
3. Floating-Point Constants- Digits followed by a decimal point and digits
4. Relational Operators- Handles both single-character (`<`, `>`) and multi-character operators (`≤`, `≥`, `=`, `≠`)
5. Arithmetic Operators- Single-character operators such as `+`, `-`, `*`, `/`, `=`

Each diagram clearly shows the start state, transitions based on input characters, accepting states, and retraction where necessary.

Implementation Approach

The lexical analyzer reads the source program from an input file character by character. Whitespace characters such as spaces, tabs, and newlines are ignored. Based on the transition diagram logic, characters are grouped into lexemes and classified into appropriate token classes.

For every recognized token, the analyzer prints the output in the format:

```
<TOKEN_CLASS, LEXEME>
```

Invalid characters that do not match any token pattern are reported as INVALID tokens.

Symbol Table Management

A symbol table is maintained to store identifiers encountered during lexical analysis. The following rules are followed:

- Only identifiers are inserted into the symbol table
- Keywords, constants, and operators are not inserted
- Duplicate identifiers are avoided
- Each entry contains the identifier name, type (**ID**), and line number of first occurrence

At the end of execution, the symbol table is displayed in a tabular format.

Testing and Output Verification

The lexical analyzer was tested using the given sample input program. The output was verified to ensure that:

- All valid tokens are correctly recognized
- Keywords and identifiers are properly distinguished
- No valid token is missed
- Invalid tokens are correctly flagged

- The symbol table contains only valid identifiers

The generated output and symbol table were found to be correct and consistent with the language specification.

Conclusion

In this experiment, a lexical analyzer for a toy programming language was successfully designed and implemented. The experiment provided practical understanding of token recognition using transition diagrams, handling of different token classes, and symbol table construction. This lab strengthens the foundation for further phases of compiler design such as syntax analysis and semantic analysis.